

增強課程 1：增強技術總覽與尋找方法

Lecture

CLAUDE.md 開發流程明訂一條鐵律：「不可修改 SAP 標準物件（S 開頭 / SAP 命名空間），只能透過 **Enhancement / BAdI / User-Exit**」。這條規則聽起來像是「不准做」的限制，但反過來看，它其實預告了 SAP 提供了一整套「合法插入自訂邏輯、又不動到標準物件本身」的官方機制——這整套機制的正式名稱是 **Enhancement Framework（增強框架）**，也就是本課程要教的東西。

為什麼不能直接改標準物件、卻又必須客製化？因為 SAP 標準程式 / 表格 / 畫面是靠 OSS Note / 版本升級持續維護的共用資產：如果直接改了標準原始碼，下次打 Support Package 或升版時，SAP 的修正檔（Note）套用到同一段程式碼可能會衝突（Modification Adjustment，SPAU 要手動處理），維護成本會隨著時間指數增加。Enhancement Framework 的設計精神是「把自訂邏輯放在 SAP 標準物件*旁邊*一個獨立的物件裡，SAP 標準物件在特定時機呼叫這個獨立物件」——原始碼本身完全沒被改動，只是多了一個「呼叫外掛」的動作，Note / 升版可以正常套用。

四大分類，依歷史演進順序（新的技術沒有完全取代舊的，舊系統跟舊物件仍在用舊技術，這是為什麼四種都要學）：

1. **Classic User-Exit（SMOD/CMOD，1990 年代技術）**：SAP 開發者在標準程式裡刻意埋一個呼叫（Function Exit 用 **CALL CUSTOMER-FUNCTION '001'**，Menu Exit / Screen Exit 類似機制），這個呼叫點本身叫一個「SAP Enhancement」（用 **SMOD** 瀏覽），使用者要用 **CMOD** 建一個「Enhancement Project」，把想用的 SAP Enhancement 指派進來，系統會自動產生一個空的 Include（命名規則通常是 **<Function Exit 名>ZZ<後綴>**），使用者在這個 Include 裡寫程式碼，最後 Activate Project。限制很明顯：一個 SAP Enhancement 只能被一個 Project 使用（獨占），不像後面的 BAdI 允許多個實作並存。
2. **Classic BAdI（NetWeaver 4.6 起，SE18/SE19）**：User-Exit 的物件導向化——SAP 開發者定義一個 **BAdI Definition**（SE18，本質是一個 Interface，強制繼承 **IF_BADI_INTERFACE**），使用者用 **SE19** 寫 **Implementation**（一個實作該 Interface 的 Class）。比 User-Exit 進步的地方：預設允許多個 **Implementation** 同時存在（除非該 BAdI 宣告成 Single Use），還可以用 **Filter** 讓不同 Implementation 依條件（如公司代碼、業務類型）分流啟用。
3. **新式 BAdI / Enhancement Spot（NetWeaver 7.0 起）**：BAdI 機制整合進統一的 Enhancement Framework 後的新一代——BAdI Definition 改掛在一個 **Enhancement Spot（ENHS）** 底下管理，Implementation 變成一種 **Enhancement Implementation（ENHO）**。這次查證發現一個有意思的現象：連 NetWeaver 4.6 時代就存在的老牌 Classic BAdI **MB_MIGO_BADI**（MIGO 交易碼的外掛點，套件 **MB**），在這套系統裡同時查得到兩種身分——舊式的 **SXSD/XD**（BAdI Definition，只能在 SE18 GUI 操作，ADT 只有唯讀 metadata stub）和新式的 **ENHS/XS**（Enhancement Spot，完整 ADT 可讀寫）。這證實了官方文件的說法：NetWeaver 7.0 升版時，系統會把舊 Classic BAdI 自動「包」一層新式 Enhancement Spot 外殼，讓舊 BAdI 也能被新框架的工具管理，兩種身分同時存在、指向同一組 Definition / Interface。
4. **Enhancement Point / Section（Explicit）與 Implicit Enhancement Point**：這是新框架額外提供、User-Exit / BAdI 都沒有的能力——**Explicit**：開發者在自己的程式裡用 **ENHANCEMENT-POINT / ENHANCEMENT-SECTION** 語法刻意留一個插入點（跟 User-Exit 概念類似，但是新框架語法，且原生支援多個 Enhancement Implementation 並存）；**Implicit**：新框架的殺手級功能——任何 **FORM / METHOD / FUNCTION / MODULE** 的開頭與結尾、以及程式最開頭/結尾，系統都自動提供一個隱式插入點，完全不需要原開發者事先宣告。這代表就連沒有預留任何 Explicit 插入點的老舊標準程式，只要是新框架涵蓋的版

本，一樣可以用 Implicit Enhancement Point (Source Code Plugin · [ENH0XHH](#)) 插入程式碼——這是「合法擴充任何標準物件」這件事在技術上真正落地的地方。

怎麼找一個交易/程式有哪些可用的增強點 (實務上常用、依情境選用)：

方法	適用分類	說明
ABAP Editor (SE38/SE80) Edit → Enhancement Operations → Show Implicit Enhancement Options	Implicit Enhancement Point	開啟後程式編輯器每個區塊前後會顯示可插入的小圖示標記，直接在畫面上點選就能建立 Source Code Plugin
SE18 (BAdI Builder : Definition) / SE19 (Implementation)	Classic / 新式 BAdI	SE18 可以用 Business Object 或關鍵字搜尋現有 Definition ； SE19 反過來可以查某個 Definition 已經有哪些 Implementation
SE20 (Enhancement Spot / Enhancement Implementation 維護)	新式 BAdI 、 Explicit/Implicit Enhancement	新框架的整合維護介面，可以瀏覽 Enhancement Spot 底下掛了哪些 BAdI Definition
SPRO (IMG) 部分節點的「Business Add-Ins」子節點	Classic / 新式 BAdI	SAP 針對特定業務流程，會在 Customizing 樹狀結構裡直接列出「這個流程相關的 BAdI 有哪些」，比自己用關鍵字瞎猜有效率
ADT quickSearch (sap-adt MCP · .claude/rules/sap-adt-mcp.md 第 2 節)	新式 BAdI / Enhancement Spot (ENHS)	已知名稱或猜測命名慣例 (如 ES_<模組>* 、 <交易碼>_BADI) 時可以直接查詢並讀出完整結構，本題事前準備會實際示範

學習目標

- 能講出 Enhancement Framework 存在的原因，並用一句話說清楚它跟「不改標準物件」這條專案規則的關係
- 能分辨四大增強分類 (User-Exit / Classic BAdI / 新式 BAdI(Enhancement Spot) / Explicit vs Implicit Enhancement Point) 在誰刻意留插入點、能不能多重實作、用什麼工具維護這三個面向的差異
- 知道 Classic BAdI 與新式 Enhancement Spot 之間「舊 BAdI 被包一層新外殼」的共存現象，理解這不是兩套互斥的技術，是同一套機制的新舊介面
- 能講出至少三種尋找「某交易/程式有哪些可用增強點」的方法，並知道各自適用哪個分類

事前準備

不需要新建任何 SAP 物件，這題是觀念總覽與工具查詢。用 [sap-adt](#) MCP 連線本系統 (client 130) 查證下面這個真實案例：

[MB_MIGO_BADI](#) (MIGO 交易碼「外部明細子畫面」用的 Classic BAdI，套件 [MB](#)，1990 年代末期就存在) 同時查得到兩種物件：

- [SXSD/XD](#) (BAdI Definition，舊式) ——URI 落在 [/sap/bc/adt/vit/wb/object_type/sxsdxd/object_name/MB_MIGO_BADI](#)，唯讀 metadata stub，跟

Search Help / T-code 同一類 GUI-only 物件

- **ENHS/XS** (Enhancement Spot · 新式) ——URI 是 `/sap/bc/adt/enhancements/enhsxs/mb_migo_badi` · 完整可讀 · GET 回應可以看到：

```
`xml <enhs:badiDefinition enhs:name="MB_MIGO_BADI" enhs:shorttext="BAI in MIGO for  
External Detail Subscreens" enhs:singleUse="false" enhs:useFallbackClass="false"  
enhs:filterLimitation="false"> <enhs:interface adtcore:name="IF_EX_MB_MIGO_BADI"/>  
<enhs:sampleClasses><enhs:sampleClass adtcore:name="CL_EXM_IM_MB_MIGO_BADI"/>  
</enhs:sampleClasses> </enhs:badiDefinition> `
```

`enhs:singleUse="false"` 證實這是允許多重實作的 BAdI；`IF_EX_MB_MIGO_BADI` 這個 Interface 讀出來可以看到 `interfaces IF_BADI_INTERFACE`。(BAdI Interface 的強制繼承) + 一串方法 (如 `CHECK_ITEM`——過帳前檢查，`ET_BAPIRET2` 收集錯誤訊息；`POST_DOCUMENT`——過帳後掛勾，可以在這裡追加自己的記錄邏輯)，對照 Interface 課程 if02 教過的「BAPI 錯誤回報用 `BAPIRET2` 結構化表格」手感，可以看出 BAdI Interface 設計上也延續了同一套慣例。

題目需求

1. 完成四大增強分類比較表 (可直接照抄 Lecture 的分類段落整理，重點是理解每一格的理由，不是死背)：分類 / 代表工具 (維護 Definition 用的交易碼) / 誰刻意留插入點 / 能不能多重實作。
2. 用 **sap-adt MCP** 或直接 **curl quickSearch**，查證 **MB_MIGO_BADI** 確實同時有 **SXSD/XD** 與 **ENHS/XS** 兩種身分 (沿用事前準備的查詢方式)，並用一句話解釋為什麼會有這個現象。
3. 情境判斷 (針對下面三個情境，先判斷該用四大分類的哪一種，再說明理由)：
 - 情境一：接手一個 2005 年上線、從未升版過核心模組的舊系統，要在採購訂單交易碼的一個既有畫面按鈕動作後追加自訂檢查
 - 情境二：現在的系統版本支援新框架，要在一支完全沒有預留任何插入點的標準 Function Module 呼叫前後插入一段記錄邏輯
 - 情境三：要設計一個全新的、允許多組客製化規則同時生效 (例如不同工廠各自客製一套邏輯) 的擴充點，且從一開始就用最新框架的能力設計
4. 找增強點練習：假設你要在交易碼 **MIGO** 找有哪些 BAdI 可用，寫出你會依序嘗試的 2~3 個方法 (可以引用 Lecture 的尋找方法表)，並說明為什麼照這個順序試。

參考答案 (情境判斷)

- 情境一：優先看有沒有 Classic User-Exit (**SMOD/CMOD**) 或 Classic BAdI (**SE18/SE19**) ——舊系統、舊模組通常還是靠這兩種舊技術留的插入點，新框架的 Explicit/Implicit Enhancement Point 是後來版本才有的能力，舊物件不會平白多出來。
- 情境二：Implicit Enhancement Point (**Source Code Plugin**) ——正是這個技術存在的理由：完全不需要原開發者事先留插入點，只要系統版本支援新框架，任何 FM 呼叫前後都有隱式插入點可用。
- 情境三：新式 BAdI / Enhancement Spot——原生支援多重實作 + Filter (可以依工廠代碼分流)，是四種裡功能最完整、最適合「從零設計」的選項；Classic BAdI 雖然也能多重實作，但新式是目前 SAP 官方建議的設計方向。

思考題

1. 如果四大分類裡新式的 (Enhancement Spot / Explicit / Implicit) 功能都比舊的 (User-Exit / Classic BAdI) 強，為什麼 SAP 沒有把舊系統的 User-Exit 全部自動轉成新式 Enhancement Spot？(提示：

User-Exit 底層的呼叫點是寫死在標準程式碼裡的 `CALL CUSTOMER-FUNCTION`，要轉換成新框架的插入點，等於要改動標準程式碼本身去換掉呼叫語法——這正好抵觸 Enhancement Framework 一開始要解決的問題「不動標準原始碼」，所以 SAP 選擇讓兩套機制並存，而不是全面遷移；只有 Classic BAdI 因為本來就是獨立的 Definition/Interface 物件，才有辦法在背後包一層新外殼而不驚動原始碼)

2. `MB_MIGO_BADI` 的 `enhs:singleUse="false"` 代表可以多重實作，但它畢竟是「Classic」年代設計的 BAdI——如果兩個團隊各自寫了一個 Implementation，會不會互相衝突？(提示：不一定衝突，但要看 Implementation 有沒有用 Filter 區分適用情境；如果兩個 Implementation 對同一個方法(如 `CHECK_ITEM`)都寫了邏輯、又沒有用 Filter 分流，執行順序可能不確定，這是多重實作 BAdI 在團隊協作時要注意的實務風險，會在 en05 深入講)
3. 承 CLAUDE.md 的專案規則——如果有一天發現某個 SAP 標準物件連 **Implicit Enhancement Point** 都沒有涵蓋到(例如某些特殊的資料庫存取邏輯)，還能怎麼客製化而不違反「不改標準物件」的規則？(提示：這代表可能要退回更早的技術，如 Classic User-Exit / Classic BAdI 看有沒有現成插入點；如果四種都沒有，SAP 官方管道通常是提交客戶事件/Incident 要求 SAP 官方新增擴充點，或改從呼叫端 / 上游流程著手客製化，而不是硬改標準物件本身)

答案

不新建任何 SAP 物件；四大分類比較表與情境判斷見本題內文。`MB_MIGO_BADI` 已用 `sap-adt` MCP 於 client 130 實際查證同時存在 `SXSD/XD`

(`/sap/bc/adt/vit/wb/object_type/sxsd/object_name/MB_MIGO_BADI`，唯讀 stub) 與 `ENHS/XS` (`/sap/bc/adt/enhancements/enhsxs/mb_migo_badi`，完整可讀，`singleUse="false"`，Interface `IF_EX_MB_MIGO_BADI` 已讀出含 `CHECK_ITEM` / `POST_DOCUMENT` 等方法) 兩種身分，證實 Classic BAdI 在新框架下的「雙重身分」現象是真實系統行為，不是文件推論。